

Table of contents

- [Introduction to QFN-48 Package, chip, pads, core, die and IPs](#)
- [Introduction to RISC-V](#)
- [From Software Applications to Hardware](#)
- [Introduction To All Components Of Open-Source Digital Asic Design](#)
- [Simplified RTL2GDS flow](#)
- [Introduction to OpenLANE and Strive chipsets](#)
- [OpenLANE Directory structure in detail](#)
- [Design Preparation Step](#)
- [Review files after design prep and run synthesis](#)
- [Steps To Characterize Synthesis Results](#)

Introduction to QFN-48 Package, chip, pads, core, die and IPs

This document introduces foundational concepts and terminology in open-source hardware and chip design, focusing on understanding electronic boards, the internal structure of chips, and the critical role of foundries. It is intended as an entry-level guide for students and newcomers to the hardware design field.

Core Concepts

Electronic Boards and Block Diagrams

- **Electronic boards** are commonly encountered in industry and learning environments, including familiar boards like those used with Arduino or

FPGAs.

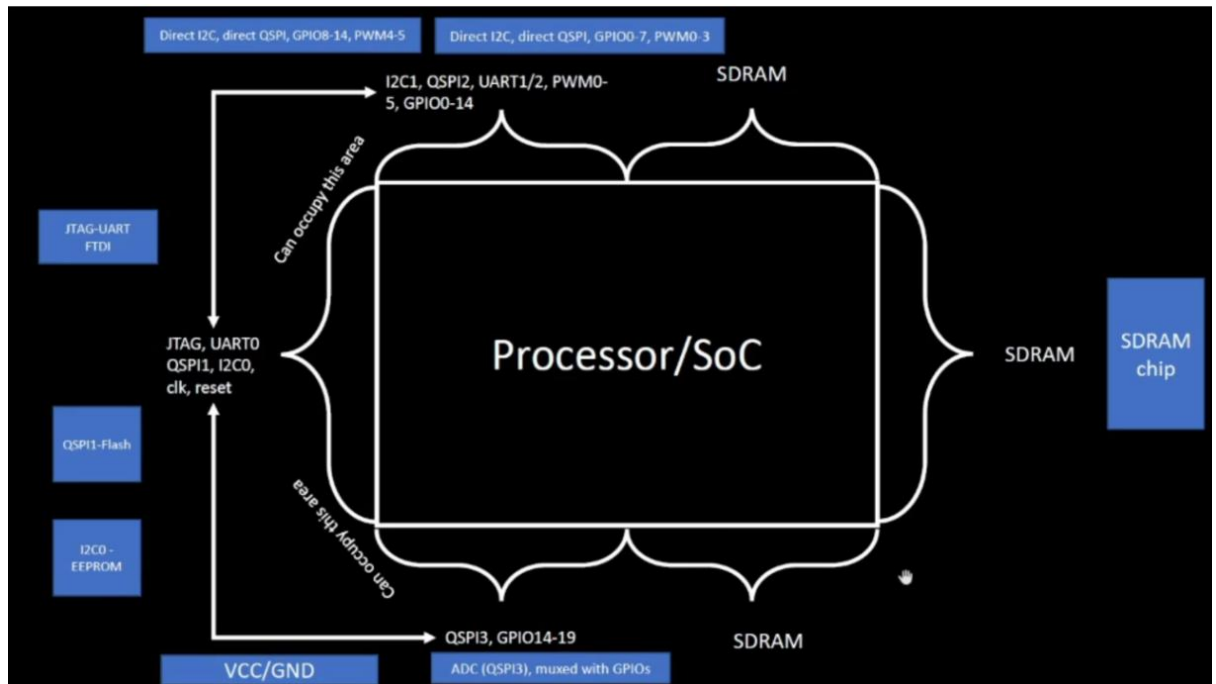


- A *block diagram* is the primary method for describing the functional organization of a board. At its center is the **processor**, surrounded by several key interfaces and components such as programming ports (e.g., JTAG), storage (e.g., QSPI flash, I2C EEPROM), power supply (VCC, GND), and analog/digital converters (ADC, DAC).

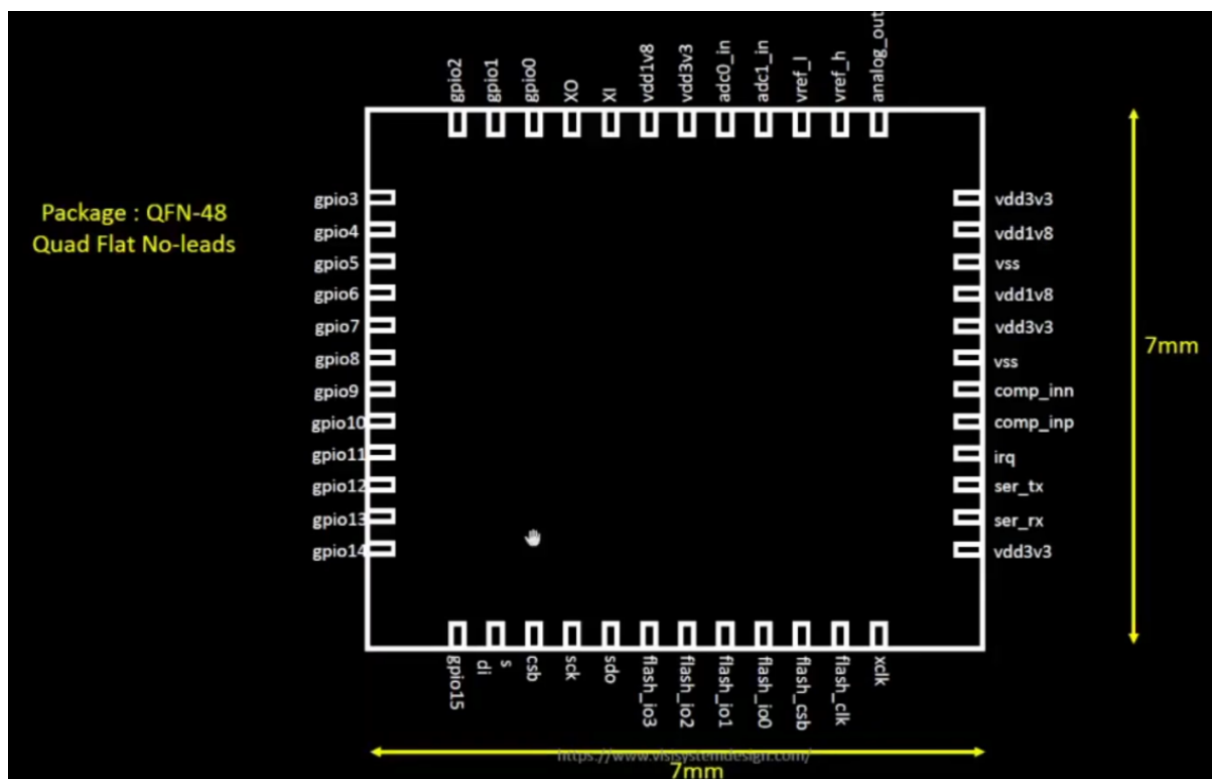
Chip, Package, and Internal Structure

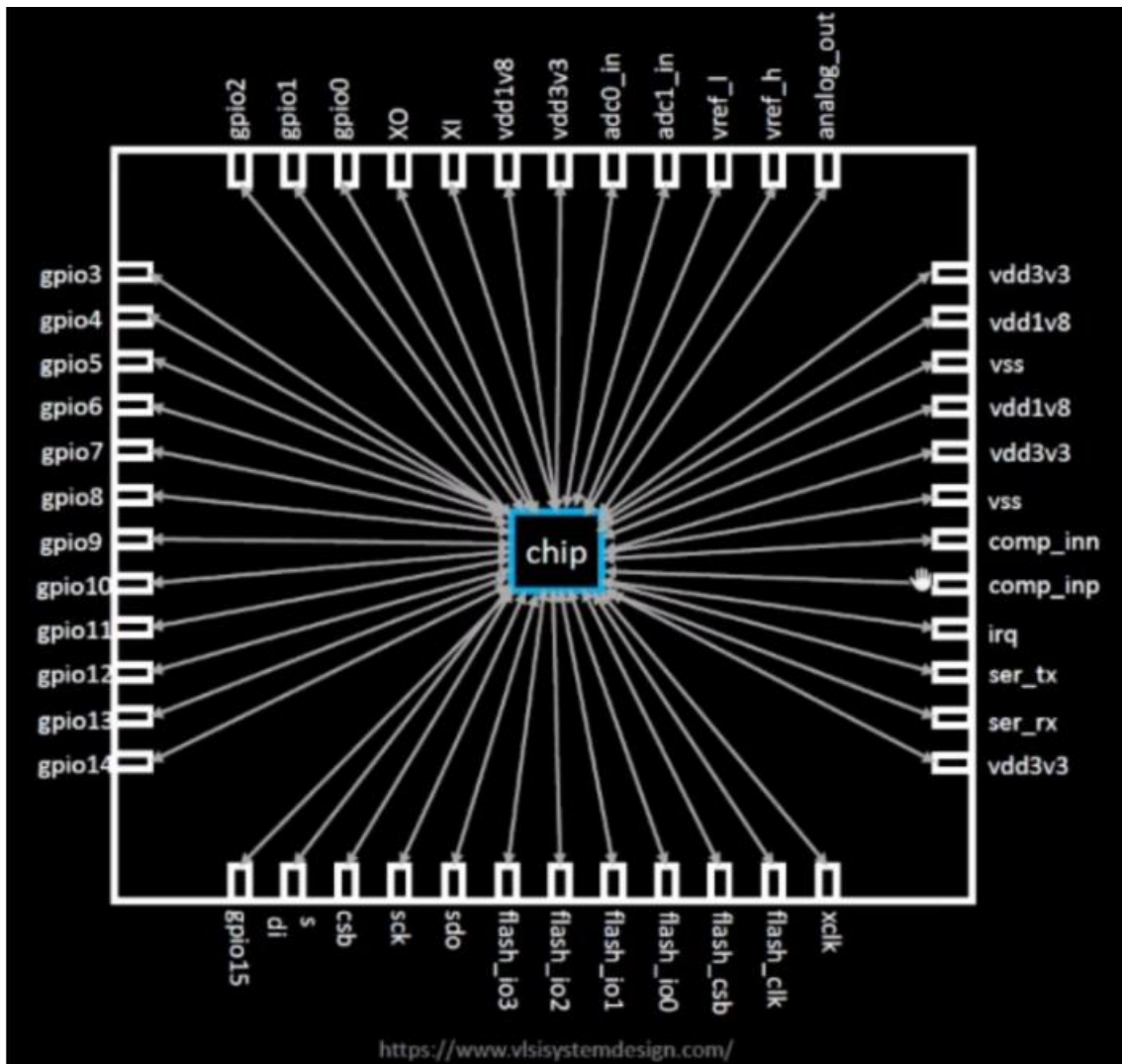
- What is often referred to as a "chip" is more precisely called a **package**. A package is the physical container (e.g., QFN, quad flat no-leads) that holds the silicon die. Packages vary by the number of pins (e.g., 48 or 256) and overall

size.



- The **chip (die)** is placed at the center of the package. Electrical connections between the package pins and the chip are usually made using **wire bonds**, which transmit signals between the external world and the internal circuitry.



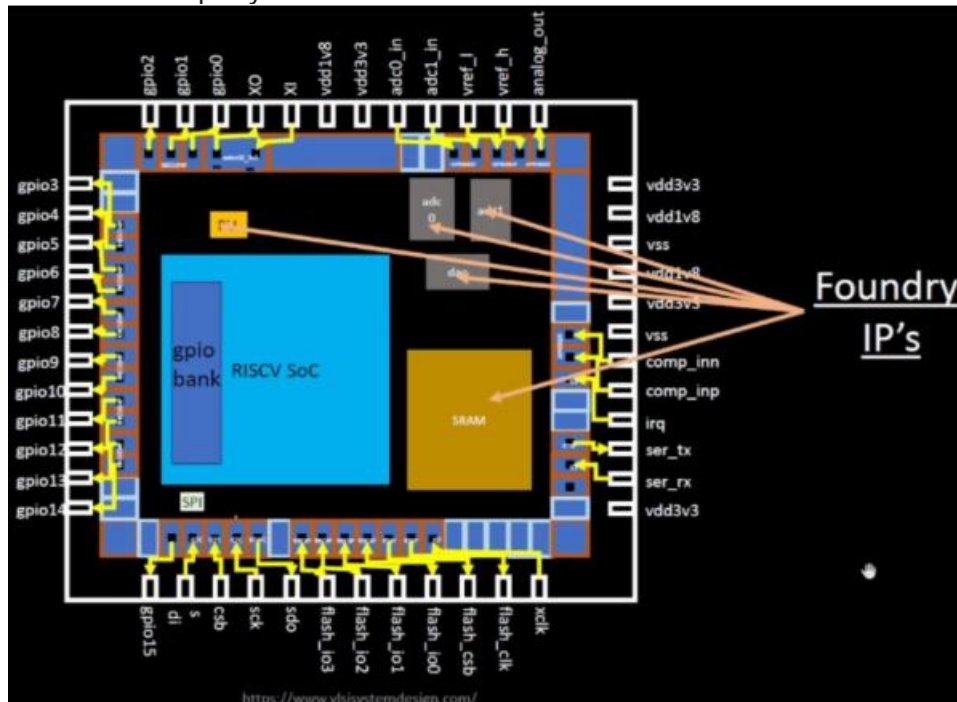


- Upon examining the chip itself:
 - **Pads:** Serve as gateways or "doors" through which signals enter and exit the chip.
 - **Core:** The central region housing the fundamental digital logic (e.g., AND/OR gates, multiplexers).
 - The chip is fabricated on a **silicon wafer**, and the area manufactured is called the *die*.

System on Chip (SoC) Organization

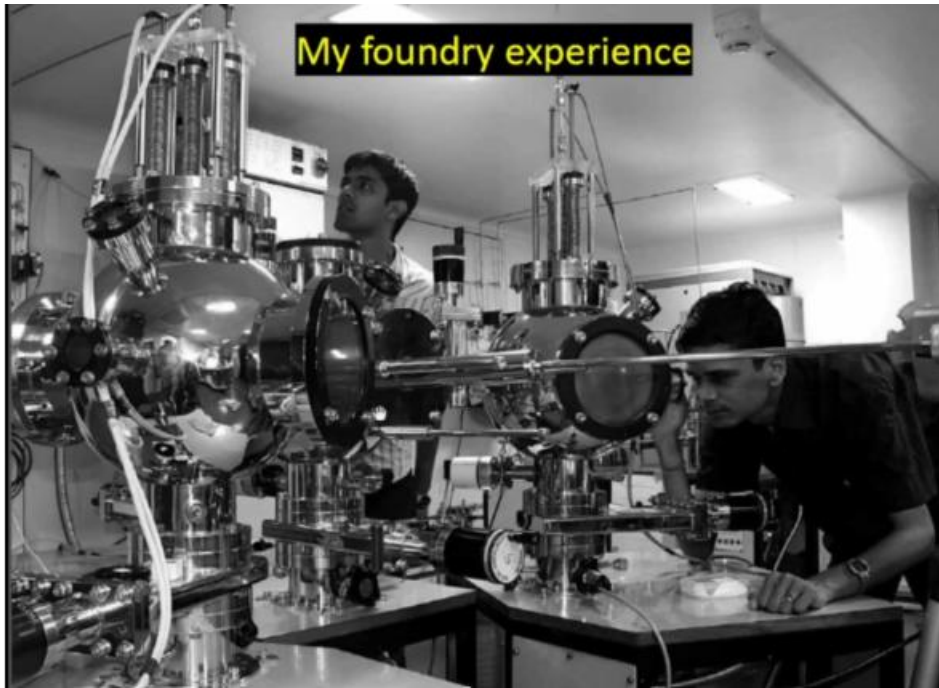
- In modern chips (e.g., those based on RISC-V), the SoC design integrates multiple components:
 - Central processor(s)

- **SRAM** (Static Random-Access Memory)
 - Communication modules (e.g., SPI controllers)
 - Clocking elements (e.g., PLL: Phase-Locked Loop)
 - Analog interfaces (ADC, DAC)
- These functional blocks are typically represented as separate areas or macros within the chip layout.



Foundry and IP (Intellectual Property) Concepts

- A **foundry** refers to the specialized manufacturing facility where chips are produced. Foundries contain deposition, lithography, and other fabrication equipment.
- **Foundry IPs** (Intellectual Properties) are pre-designed and verified blocks (such as ADCs, DACs, memory modules) provided by the foundry for integration into custom chips. These blocks require significant engineering expertise to create and are shared as reusable intellectual property elements.
- The distinction between "macros" and "IPs":
 - **Macros:** Typically refer to standard digital logic blocks.
 - **IPs:** More complex, often mixed-signal or specialized circuits, requiring advanced design knowledge.
- Effective chip design requires continuous communication with the foundry, typically through standardized interface files provided by the foundry,



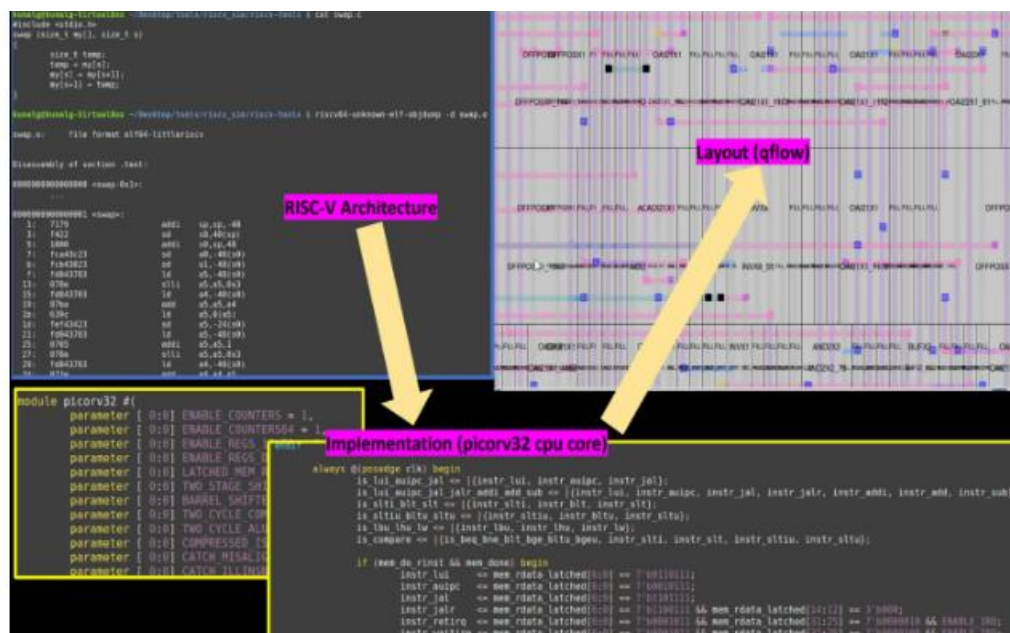
- enabling the translation of design intent into manufacturable hardware.

Key Points

- Understanding **board layouts** and **block diagrams** is foundational before delving into chip-level design.
 - The term **package** refers to the entire external housing of the chip, not just the silicon die.
 - **Wire bonds** and **pads** are physical mechanisms for interfacing between the external board and internal chip logic.
 - The **core** of the chip contains the main processing and logical functions; the periphery involves communication and I/O interfaces.
 - **System on Chip (SoC)** architecture groups essential components (memory, analog blocks, communication modules) for integrated functionality.
 - **Foundry and IP** are central to modern chip design, with foundries supplying both the physical manufacturing capability and foundational building blocks for design integration.
 - Successful hardware design and fabrication rely on standardized communication between designers (engineers) and foundry operations, often mediated by shared interface files and design standards.
 - Mastery of these fundamentals opens up advanced work in hardware design, SoC integration, and semiconductor manufacturing.
-

Introduction to RISC-V

This introduction explains the foundational role of the **Instruction Set Architecture (ISA)**—focusing on RISC-V—in computer hardware and software design. ISA serves as the central language between software applications and hardware, outlining how instructions are formatted, interpreted, and executed within a processor. The flow from high-level application code to executable machine binaries involves several stages and interfaces, ultimately allowing programs to interact with physical silicon implementations.



Core Concepts

Instruction Set Architecture (ISA)

- The **ISA** is the formal definition of how a computer's processor executes instructions and manages operations. It acts as an interface dictating how software communicates with hardware.
- In RISC-V and similar architectures, the ISA specifies available operations, data types, register sets, and memory addressing mechanisms.
- The ISA does not exist physically within computer memory; rather, it is a **conceptual specification** that hardware designs implement.

Compilation Flow

- Applications (e.g., in C) must be translated so that processors can execute them. The flow is:
 - **Source Code** (C, etc.) → **Assembly Language** (RISC-V assembly) → **Machine Code** (binary instructions)
 - Machine code is composed of binary representations: strings of 0 and 1; sometimes expressed as hexadecimal for readability.
- The translation process ensures that the logic authored by the programmer becomes a stream of hardware-level cmds.log.

RTL and Hardware Implementation

- The **ISA** forms the basis for hardware design.
 - Design engineers use **Hardware Description Languages (HDLs)** to implement the ISA specifications as **Register Transfer Level (RTL)** logic.
 - RTL abstracts the internal data flows and logic required to realize the operations defined by the ISA on silicon.
- The **RTL implementation** is then synthesized to produce the physical layout of the chip (using standard chip design flows like Place-and-Route).

Key Points

- **ISA as a Language:** The ISA can be likened to the language that software uses to communicate intent to the hardware—defining the permissible set of instructions for computation and control.
- **Separation of Concerns:** The ISA allows hardware designers and software developers to operate independently, provided both conform to the ISA specification. This separation ensures software binary compatibility across different hardware implementations adhering to the same ISA.
- **Binary Execution:** The final code executed by hardware consists of sequences of binary digits (bits) that encode the instructions as specified by the ISA. This is the format directly understood by the chip's logic gates and transistors.
 - Example: If a program swaps two numbers, that logic is ultimately converted to a specific set of binary instructions implemented and executed by the chip.
- **ISA vs. Microarchitecture:**

- **ISA:** High-level, abstract specification ("what" the hardware should do).
- **Microarchitecture/RTL:** Actual realization ("how" those operations are implemented) in silicon.
- **Hardware Description Language (HDL):** Used to express RTL implementations that embody the ISA specifications in physical logic.
- **Design Flow Overview:**

i. Start with ISA specifications

- ii. **Implement with HDL/RTL**
- iii. **Synthesize RTL into chip layout (physical design)**
- iv. **Applications compile to binary executable for that ISA**
- v. **Chip executes machine instructions, producing output for the user**
- **User Perspective:** For most users, executing a program appears seamless. The underlying complexity—translating software logic through layers of abstraction down to transistor-level operations—remains hidden.

From Software Applications to Hardware

This documentation focuses on the theoretical aspects of how application software interacts with hardware through the system software layer. It explains the flow from application software to being converted into binary instructions that the hardware can understand. Key components include the operating system, compiler, and assembler, each playing a crucial role in this process.

Core Concepts

System Software Layers

- **Operating System (OS):** Serves as the interface between application software and hardware. It handles memory management, I/O operations, and ensures the software runs smoothly by allocating necessary resources. The OS converts application programs into assembly language and then binary code for the hardware.
- **Compiler:** Translates source code (e.g., C or Java) into instructions based on the hardware architecture (e.g., Intel X86, ARM, MIPS).

- **Assembler:** Converts assembly language into machine code (binary digits) that the hardware can execute directly.

Instruction Set Architecture (ISA)

- **Definition:** Acts as an abstract interface between the programming language and hardware, defining how instructions are executed.
- **Function:** Provides a set of instructions that the hardware understands, allowing application software to communicate with the hardware. Examples include X86, ARM, and MIPS ISA.

Hardware Description Language (HDL)

- **Definition:** Used to describe digital circuits at a high level, before they are synthesized into gates and flip-flops.
- **Function:** Essential for designing hardware that can interpret and execute the binary instructions derived from the ISA.

Key Points

- **System Flow:** Application software enters the system software, which converts it into assembly and then binary code. This binary code is executed by the hardware.
 - **Hardware vs. Software Interaction:** The compiler ensures that the output instructions are compatible with the specific hardware architecture.
 - **RISC vs. CISC Architectures:** RISC (Reduced Instruction Set Computing) focuses on simpler instructions executed quickly, while CISC (Complex Instruction Set Computing) involves more complex instructions.
 - **Synthesis and Implementation:** HDL specifications are synthesized into gate-level netlists, which are then physically implemented on the chip.
-

Introduction To All Components Of Open-Source Digital Asic Design

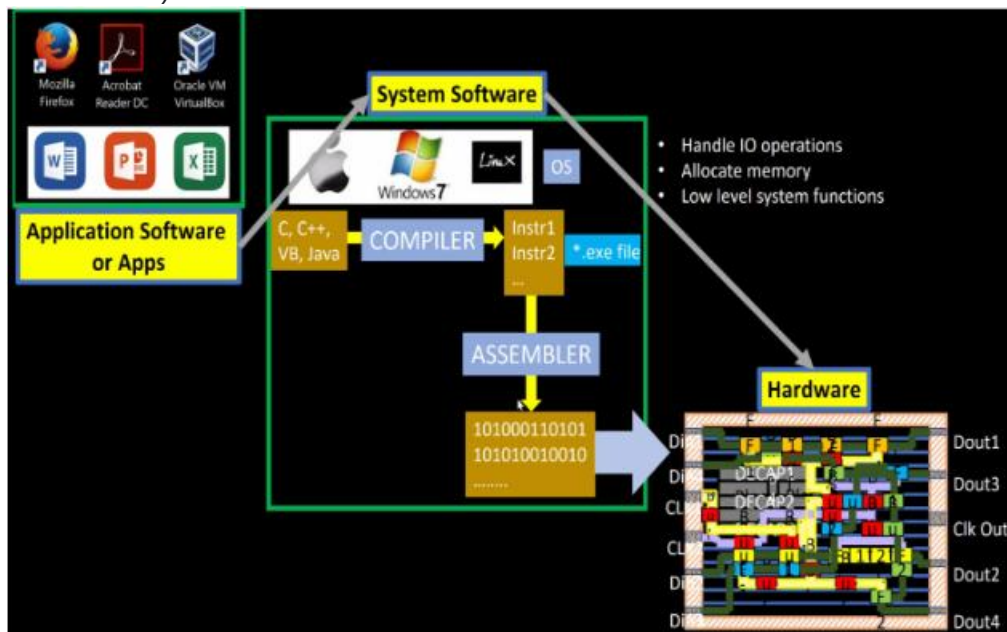
Open Lane is an open-source, Python-based platform designed to automate the flow of digital Application-Specific Integrated Circuit (ASIC) implementation, supporting customizable workflows and integration of various open-source Electronic Design Automation (EDA) tools. The platform facilitates the entire ASIC design process, from high-level hardware descriptions down to manufacturing-ready physical layouts, and is a major enabler for open-source silicon development. The recent release of open-source Process Design Kits (PDKs), such as the SkyWater 130nm PDK, completes the open-source toolchain by providing essential fabrication technology data to the community.

Core Concepts

ASIC Design Flow

- **ASIC (Application-Specific Integrated Circuit) design** is a methodical process transforming functional hardware descriptions into manufacturable chip layouts.
- The flow starts from RTL (Register-Transfer Level) models, written in hardware description languages (HDLs) like Verilog or SystemVerilog, and progresses through stages like synthesis, floorplanning, placement, routing, and verification.
- Multiple EDA tools are orchestrated to automate these steps, resulting in a final GDSII file suitable for fabrication.
- Open Lane uses integrated open-source tools such as Yosys (for synthesis), OpenROAD (for physical design), Magic (for layout), and Klayout (for

verification).



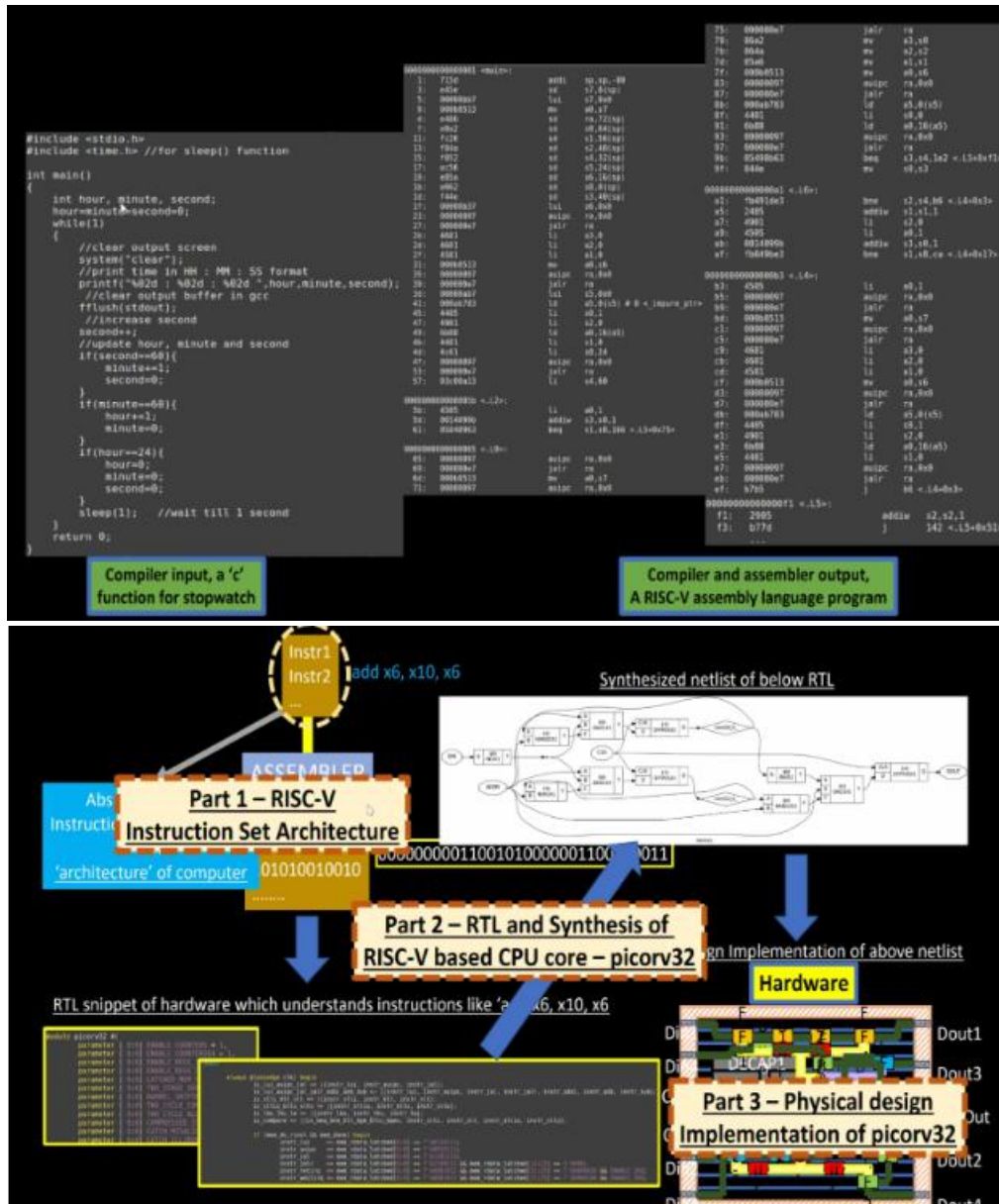
Process Design Kit (PDK)

- A **PDK (Process Design Kit)** is a comprehensive bundle of device models, design rules, library files, and technology-specific data that enables designers to create circuits compatible with a specific fabrication process.
- Historically, PDKs were proprietary, limiting access and participation in silicon fabrication to select organizations.
- The recent release of open-source PDKs—for example, the SkyWater 130nm PDK—now allows broader access, so all three foundational elements of ASIC design (RTL, EDA tools, PDK) can be open-source.
- The PDK encapsulates models such as transistor characteristics and standard cell libraries, essential for accurate simulation and layout.

Technology Nodes and Market Relevance

- **Technology node** refers to the minimum feature size of transistors in an integrated circuit, commonly expressed in nanometers (e.g., 130nm, 5nm).
- While 130nm node is no longer cutting-edge, it remains commercially viable, representing a significant market share and supporting applications that do not require the extreme performance offered by the latest nodes.

- Lower node sizes (5,nm, 7,nm) allow for higher density and performance but are costly; 130nm offers a balance between performance and affordability.



Key Points

- **Open Lane platform** provides an automated, customizable infrastructure for converting RTL designs into manufacturable layouts, lowering the barrier to ASIC development.
- The **SkyWater 130nm PDK** enables true open-source silicon fabrication, giving designers essential process data for successful chip design and tape-out.
- ASIC design involves these key elements:

- Hardware description: typically at RTL, via HDLs like Verilog/SystemVerilog.
 - EDA tools: automate synthesis, placement, routing, verification, and more.
 - PDK: supplies foundry-specific fabrication data, such as device models and design rules.
 - ASIC design methodology is encapsulated in a "flow," a sequence of steps that implements the design from RTL to the final physical layout (GDSII format).
 - The flow manages complex inter-tool orchestration and applies hierarchical design protocols.
 - Equations such as gate delay, power, and area are essential throughout design stages. For example:
 - Gate delay: $t_{pd} = k \cdot L^2 / (V_{dd} - V_{th})$
 - Dynamic power: $P = \alpha C V_{dd}^2 f$
 - Where L is channel length, V_{dd} is supply voltage, V_{th} is threshold voltage, C is capacitance, f is frequency.
 - Open Lane and similar open-source flows accelerate innovation, reduce cost, and democratize access to advanced silicon design.
 - 130nm technology is proven in high-performance chips (e.g., Pentium 4 running at 3.5GHz) and remains relevant for many practical applications.
 - With all three elements—open-source RTL, EDA, and PDK—now available, the creation of 100% open-source ASICs is feasible for a broad range of designers and organizations.
-

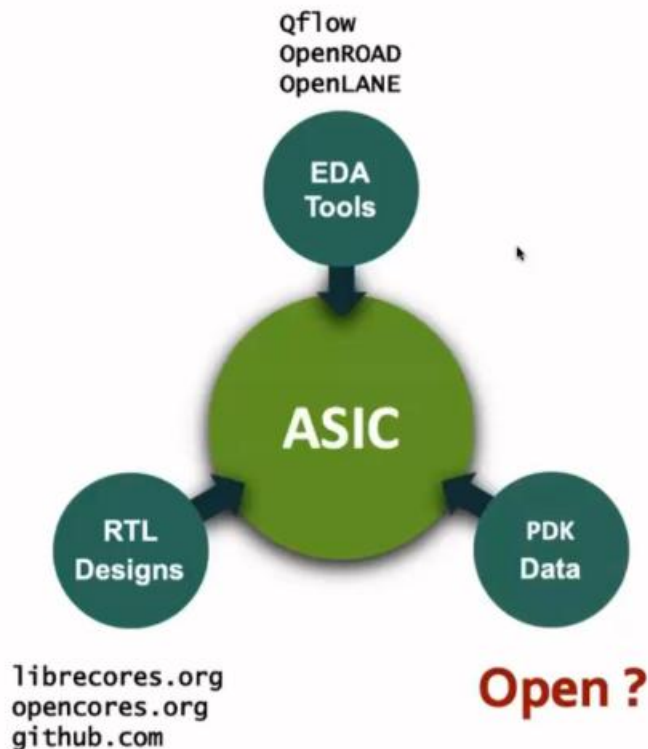
Simplified RTL2GDS flow

This document outlines the theoretical foundation of the **ASIC (Application-Specific Integrated Circuit) design flow**, detailing each stage from the initial RTL model to final mask layouts in GDSII format. It covers conceptual steps such as synthesis, floorplanning, placement, clock tree synthesis, routing, and verification, emphasizing how each phase contributes to the manufacturable silicon chip.

Core Concepts

ASIC Design Flow Overview

ASICs are custom-designed integrated circuits tailored for specific applications, offering benefits like high performance, power efficiency, and reduced size. The design process is a sequence of well-defined, interdependent stages ensuring the circuit meets its intended function and manufacturability.



Register Transfer Level (RTL) and Gate-Level Netlist

- **RTL Model:** The chip's functionality is described in a hardware description language (HDL) such as Verilog or VHDL, representing the data flows and operations at the register level.
- **Synthesis:** The RTL model is transformed into a gate-level netlist—a structural description of the design using components (standard cells) from a technology library. This netlist is functionally equivalent to the RTL specification.
- **Standard Cells:** Each standard cell has a regular (fixed-height, variable-width) physical layout and is described by multiple views used by electronic design automation (EDA) tools, such as:
 - **Liberty view:** Contains timing and power information.

- **HDL behavioral view:** For simulation.
- **SPICE/CDL view:** For circuit-level simulation.
- **Physical layout views:** Include abstract and detailed layouts.

Floorplanning and Power Planning

- **Floorplanning:** Defines the spatial organization of the chip or macros, partitioning the die into areas for different components and establishing the foundation for layout. Macro dimensions and locations are set, and placement rows/routing tracks are defined.
- **Power Planning:** Includes designing the power distribution network (PDN) to deliver power to all chip components. This network uses multiple VDD and ground pins, rings, and metal straps—primarily on upper (thicker, lower resistance) metal layers—to minimize resistance and voltage drop (IR drop) and mitigate electromigration.

Placement

- **Placement:** Involves assigning physical locations to each cell in the design. Connected cells are placed close together to minimize signal delay and facilitate routing.
 - **Global Placement:** Finds approximate locations, potentially allowing overlaps.
 - **Detailed Placement:** Adjusts cell locations to ensure legal, non-overlapping placement.

Clock Tree Synthesis

- **Clock Tree Synthesis (CTS):** Builds the clock distribution network to deliver the clock signal with minimal skew (timing difference between clock arrival at various points) and low latency. The network is tree-shaped, with various structures (e.g., H-tree, X-tree, or hybrids) used to meet timing requirements.

- **Skew Minimization:** Minimizing clock skew is critical for correct sequential logic operation, ensuring synchronous components receive the clock with minimal timing differences.

$$skew = t_{clk,max} - t_{clk,min}$$

Signal Routing

- **Routing:** Establishes the actual metallic connections between placed cells using the available set of metal layers (as specified by the process design kit, or PDK).
 - **Global Routing:** Generates high-level routing guides using coarse grids.
 - **Detail Routing:** Implements specific wire paths and via connections, following design rules.
- **Metal Layers:** Each has defined thickness, pitch (track spacing), and minimum width.
- **Routing Challenges:** Finding valid wiring patterns within layer/track constraints and minimizing wirelength, congestion, and parasitics.

Layout and Verification

- **Final Layout:** The complete geometrical description of all circuit features, generally captured in GDSII format, ready for mask generation.
- **Design Rule Checking (DRC):** Ensures layout complies with manufacturing constraints (such as minimum width and spacing).
- **Layout Versus Schematic (LVS):** Verifies that the physical layout matches the intended schematic or netlist.
- **Static Timing Analysis (STA):** Performs timing verification to ensure all paths meet required timing, confirming the circuit will operate correctly at the target frequency.

$$T_{clk} \geq \max(\text{setup time, combinational delay, hold time})$$

Tape-Out

- **Tape-Out:** The conclusion of the implementation flow, producing the final mask set layouts for fabrication. "Tape-out" historically referred to physically sending magnetic tapes with the design data to the foundry.

License - **Apache 2.0**

June 30, 2020



FOSS 130nm Production PDK

github.com/google/skywater-ndk

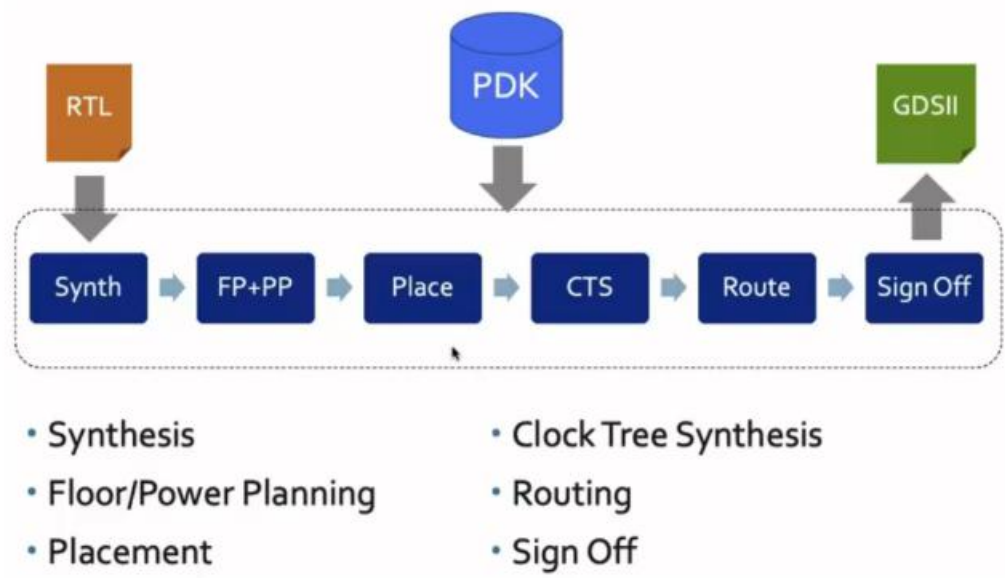
Key Points

- The ASIC design flow is a sequential, interdependent process including specification, RTL modeling, functional verification, synthesis, floorplanning, placement, clock tree synthesis, routing, layout verification, and tape-out.
 - Standard cells are modular building blocks with predefined electrical and physical characteristics, simplifying synthesis and placement.
 - Power and clock distribution networks are crucial for circuit reliability, necessitating careful planning to reduce IR drop, electromigration, and timing mismatches.
 - Verification occurs at multiple stages to guarantee functionality (functional and layout-vs-schematic checks), manufacturability (design rule checks), and timing compliance (static timing analysis).
 - The entire design is captured in GDSII format for mask generation and manufacturing. Successful ASICs require close attention to detail at each stage to meet the design's functional, physical, and performance goals.
-

Introduction to OpenLANE and Strive chipsets

Open-source ASIC development leverages publicly available Process Design Kits (PDKs) and design automation flows to create custom chips. The reference methodology described is called **Open Lane**, which is an open-source toolchain for ASIC implementation designed for high automation, minimal human intervention, and broad compatibility with open silicon platforms. It targets the SkyWater 130nm process and supports multiple chip architectures, memory configurations, and modes of operation. The overall objective is to enable fully automated and verifiable chip design from RTL to final layout, with emphasis on design space exploration and transparent licensing.

Core Concepts



ASIC Design Flow

The ASIC (Application-Specific Integrated Circuit) design flow is a systematic process that transforms a functional concept into a physical chip. It generally proceeds through these stages:

- **Specification and Architecture:** Define the chip's purpose, performance targets, and architectural blocks, including memory, control, and datapaths.
- **RTL Design:** Functional description is written in a hardware description language (HDL) at the Register Transfer Level (RTL), outlining how signal flows between registers and functional blocks.

- **RTL Verification:** Simulation tools confirm that the RTL code performs as intended. Verification includes functional correctness and testbench-driven coverage analysis.
- **Synthesis:** Convert RTL to a gate-level representation, mapping functional code to standard cell libraries corresponding to the chosen fabrication process.
- **Gate-Level Verification:** Ensure synthesized netlist matches the original specifications and functions under intended timing constraints.
- **Place and Route (P&R):** Physically arrange gates and route interconnections, perform clock tree and power grid synthesis.
- **Physical Verification:** Validate that the layout follows design rules (DRC), matches intended circuit topology (LVS), and is free from manufacturing violations.
- **Timing Analysis:** Confirm the design is free of timing violations, i.e., meets setup and hold time constraints for all paths:
- **Tapeout:** Produce the final layout file (GDSII) and send to manufacturing.

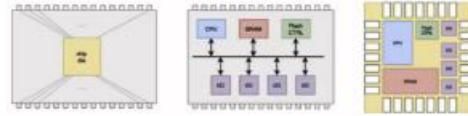
Open Source ASIC Methodology

Open Lane provides an open-source reference implementation for the ASIC flow. Key aspects include:

- Integration with open-source PDKs such as SkyWater 130nm.
- Containerized deployment for out-of-the-box operation.
- Support for both autonomous (push-button) and interactive design flows.
- Enables design and hardening of macros as well as whole chips.

- Provides extensive design space exploration tools for discovering optimal

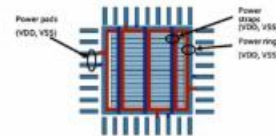
- **Chip Floor-Planning:** Partition the chip die between different system building blocks and place the I/O Pads



- **Macro Floor-Planning:** Dimensions, pin locations, rows definition

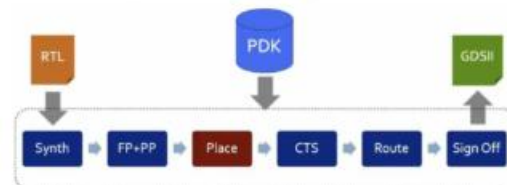


- **Power Planning**



Placement

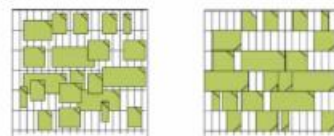
- **Placement** involves assigning physical locations to each cell in the design. Connected cells are placed close together to minimize signal delay and facilitate routing.
 - **Global Placement:** Finds approximate locations, potentially allowing overlaps.
 - **Detailed Placement:** Adjusts cell locations to ensure legal, non-overlapping placement.



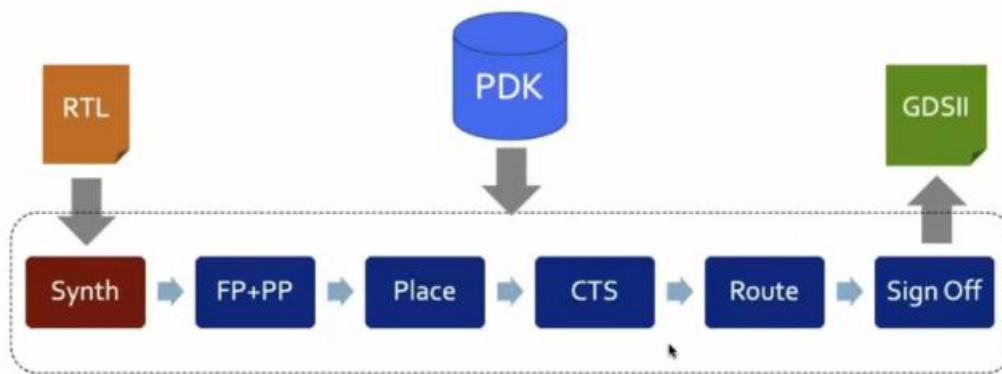
- **Place the cells on the floorplan rows, aligned with the sites**



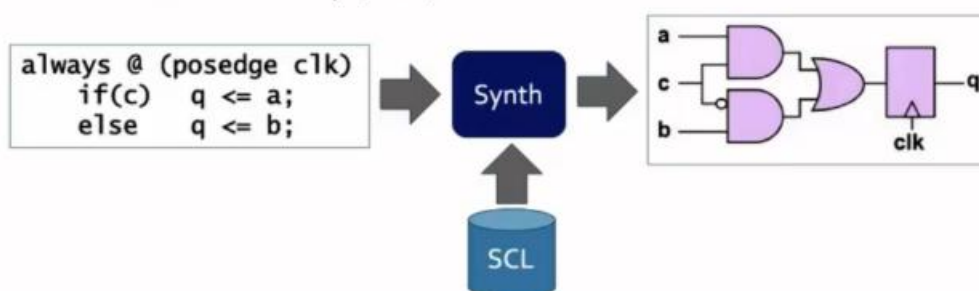
- **Usually done in 2 steps: Global and Detailed**



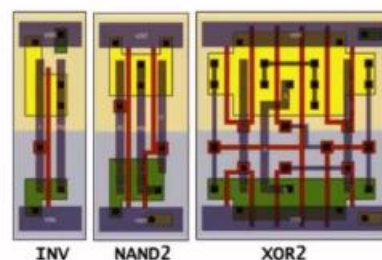
configuration settings automatically.



- Converts RTL to a circuit out of components from the standard cell library (SCL)



- “Standard Cells” have **regular** layout
- Each has different views/models
 - Electrical, HDL, SPICE,
 - Layout (Abstract and Detailed)
 - ...



Design Space Exploration

Design space exploration is a methodology for optimizing flow configuration parameters to achieve best results for timing, area, power, or other design goals. It involves:

- Sweeping a range of configuration values.
- Evaluating each candidate using metrics like timing slack and violation absence.
- Searching for the set of parameters $(P_1, P_2, \dots, P_n)$ maximizing design quality under constraints:
 - $\max Q(P_1, P_2, \dots, P_n) | V_{DRC} = 0, V_{LVS} = 0$
 - Where Q is a quality metric (timing, area, etc.)
 - V_{DRC} and V_{LVS} denote number of DRC and LVS violations.

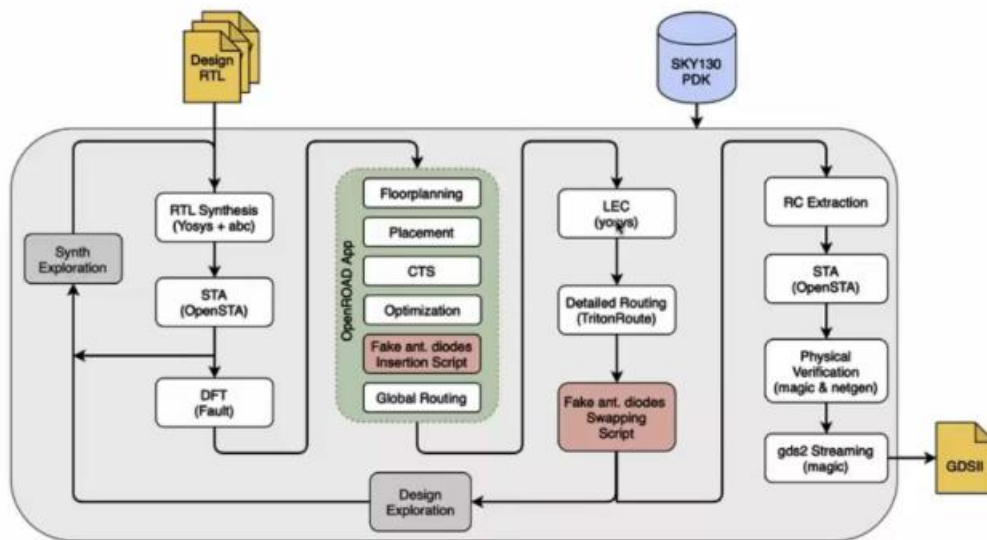
Key Points

- **Automation:** Open Lane seeks full automation from RTL to GDSII with minimal manual intervention, aiming for 'clean' tapeout-ready designs.
 - **Verification:** The flow continuously checks for violations:
 - No Layout Versus Schematic (LVS) violations.
 - No Design Rule Check (DRC) violations.
 - No timing violations, although timing closure remains a work in progress.
 - **Open Ecosystem:** All components are open, including PDKs, EDA tools, RTL libraries, and design examples.
 - **Reference Implementations:** The Thrive family showcases different memory configurations, hierarchy strategies, and library choices:
 - Multiple SRAM block sizes and memory banks.
 - Varied hierarchical organization for experimentation.
 - Design-for-test structures for manufacturability.
 - **Modes of Operation:**
 - *Autonomous:* Fully automated, push-button flow yielding final layout and reports.
 - *Interactive:* Step-wise execution for research, debugging, or custom flow tuning.
 - **Community Contributions:** The flow includes over 40 publicly available designs with best-known configuration sets, regularly updated and expanded by contributors.
-

Introduction to OpenLANE detailed ASIC design flow

The OpenLane ASIC flow enables the automatic transformation of digital designs from RTL descriptions to final physical layout formats suitable for fabrication, such as GDSII. This process utilizes open-source tooling and project collaboration to ensure accessibility and customization for a range of hardware projects. The flow incorporates specialized utilities for design and synthesis exploration, allowing for optimization across multiple metrics and design goals.

Core Concepts



End-to-End OpenLane Flow

- **OpenLane** is constructed from several open-source projects and tools, orchestrating a complete ASIC implementation flow starting from RTL (Register Transfer Level) design and ending with a finalized GDSII layout file suitable for manufacturing.
- The process relies on a Process Design Kit (PDK) that provides fabrication-specific technology data and libraries.

OpenROAD

Magic VLSI Layout Tool

KLayout

Fault

Yosys

QFlow

ABC

RTL Synthesis and Optimization

- The flow begins with **RTL synthesis**, where high-level hardware descriptions are converted into gate-level logic circuits composed of generic components.
- This synthesized logic is mapped onto a set of technology-specific cells from a standard cell library.
- The optimization stage utilizes different strategies (guided by ABC scripts), targeting objectives such as minimizing area or delay.
- Synthesis exploration utilities allow analysis of how various strategies influence design metrics like delay (t_{delay}) and area (A_{design}).

A_{design} ,

t_{delay} , metrics $\rightarrow$ synthesis strategies

Design and Synthesis Exploration

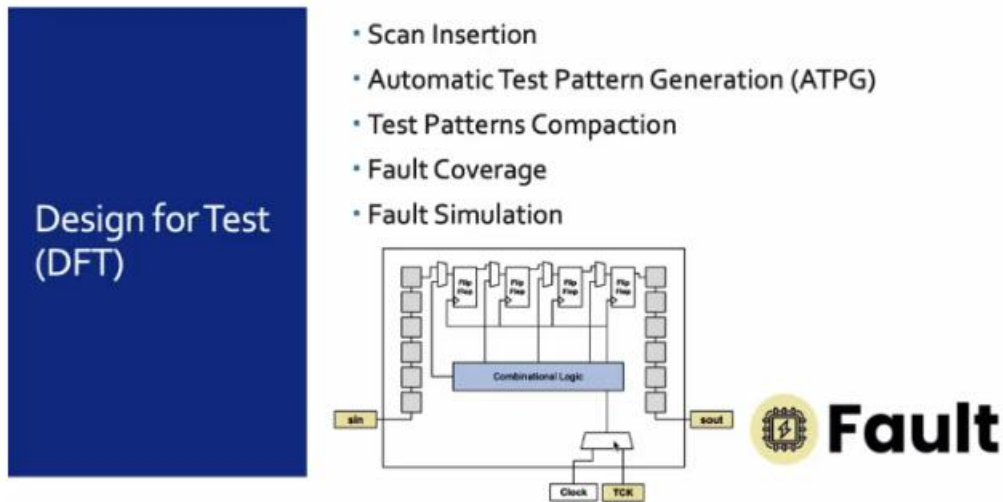
- **Design exploration utilities** enable sweeping through more than 16 possible configuration parameters, generating comprehensive metric reports (over 35 distinct metrics) and identifying violation counts for each configuration.
- These reports are essential for selecting optimal configurations and are also used in regression testing to support continuous integration for large design sets.

Design	Runtime	Cell Count	TR Vios	FP_CORE_UTIL	ROUTING_STRATEGY	GLB_RT_ADJUSTMENT
aes	1h29m8s	22932	1	40	1	0.05
aes	1h34m31s	22932	2	30	1	0.05
aes	1h41m14s	22932	9	40	1	0.05
aes	1h47m14s	22932	1	45	1	0.05
aes	1h44m14s	22932	1	40	1	0.05
aes	1h47m59s	22932	1	45	1	0.05
aes	1h49m7s	22932	1	45	1	0.05
aes	1h43m54s	22932	2	30	1	0.05
aes	1h42m58s	22932	8	30	1	0.05
cordic	0h10m51s	8275	0	45	0	0.15
cordic	0h10m35s	8275	0	45	0	0.15
cordic	0h9m55s	8275	2	40	0	0.15
cordic	0h11m25s	8275	0	45	0	0.15
cordic	0h10m3s	8275	0	30	0	0.15
cordic	0h11m6s	8275	4	40	0	0.15
cordic	0h11m3s	8275	4	40	0	0.15
cordic	0h10m25s	8275	0	30	0	0.15
cordic	0h10m26s	8275	3	30	0	0.15

Testing Structure Insertion

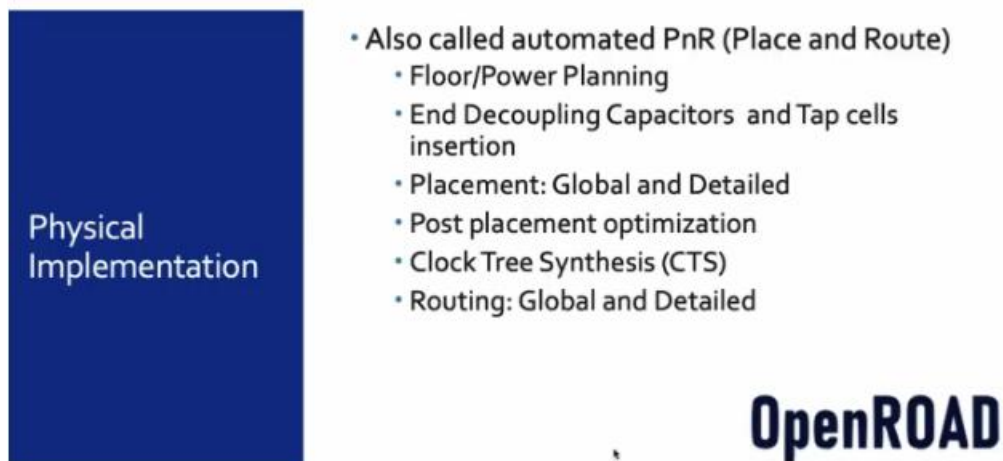
- To make designs testable after fabrication, optional steps can be enabled that insert scan chains, test pattern generation, fault coverage analysis, and, optionally, JTAG controllers for standardized testing access.

- The scan chain forms a path through which each flip-flop can be accessed and controlled for post-silicon verification.



Physical Implementation

- This phase involves **floorplanning**, macro and cell placement, power network creation, clock tree synthesis, placement optimization, and both global and detailed routing.
- Tools automate layout tasks: arranging cells, connecting signals, synthesizing clogs, and resolving placement for optimal timing and area.



Logic Equivalence Checking

- After optimizations and transformations during physical realization, **logic equivalence checking** ensures the netlist produced by physical design matches the original synthesized logic in functionality.

- This verification is crucial to avoid unintended changes introduced during physical optimization steps.



- Every time the netlist is modified, verification must be performed
 - CTS modifies the netlist
 - Post Placement optimizations modifies the netlist
- LEC is used to formally confirm that the function did not change after modifying the netlist

Yosys

Antenna Effect Handling

- Long metal wire segments may act as antennas during fabrication, collecting charge that risks gate damage. There are two mitigation strategies:
 - **Bridging technique:** temporarily connects the wire to higher metal layers.
 - **Insertion of antenna diodes:** allows charge to dissipate, protecting transistor gates.
- OpenLane includes a preventive approach by inserting a "fake" antenna diode cell during initial placement, replaced by a real diode if violations are discovered in later checks.

$$Q_{antenna} > Q_{safe} \rightarrow \text{insert diode}$$

Signoff Analysis

- **Static timing analysis (STA)** verifies the design meets timing constraints, highlighting violations if paths exceed allowable delays.
- **Design Rule Checking (DRC)** and **Layout vs. Schematic (LVS)** validation ensure the physical layout adheres to fabrication and logical requirements.
- Extraction of interconnect resistance and capacitance (R_{int} , C_{int}) is performed for accurate timing verification.

$$T_{path} \leq T_{max}; \text{ DRC pass; LVS pass}$$

Key Points

- **OpenLane automates the full ASIC flow** from RTL input to final GDSII output, leveraging open-source EDA tools.
- **Synthesis and design exploration utilities** facilitate selection of optimal configuration parameters, improving area, timing, and minimizing violations.
- **Testing structure insertion** enhances post-fabrication testability, implementing scan chains and optional JTAG.
- **Physical implementation** sequences encompass floorplan, placement, CTS, routing, and specialized optimization steps.
- **Logic equivalence checking** is critical to guarantee functional preservation after physical transformations.
- **Antenna effects** are handled through bridging or diode insertion; OpenLane includes automated approaches to insert and replace diodes as needed.
- **Signoff analysis** is performed with open-source tools, including static timing analysis, DRC, and LVS, ensuring readiness for manufacturing.

.